K.RAMAKRISHNAN COLLEGE OF TECHNOLOGY

(Autonomous)

SAMAYAPURAM, TRICHY-621 112



Incubating Minds,
Catalyzing Careers

DEPARTMENT OF INFORMATION TECHNOLOGY

Practical Record Note

Name : K Varshini

Register No 250381120522058

Course code : CGI1242

Course name : Data Science and Statistical Analysis laboratory

Branch : B.Tech – Information Technology

K.RAMAKRISHNAN COLLEGE OF TECHNOLOGY

(Autonomous)

SAMAYAPURAM, TRICHY-621 112

Incubating Minds,
Catalyzing Careers

CERTIFICATE

Certified that this is a bonafide record of work done by **K Varshini** of 3rd Semester in CGI1242 Data Science and Statistical Analysis laboratory during the academic year 2026-2027

His/her University Register Number is **2503811720522058**

Staff In-charge: **Head of the Department**

Date: Date:

Submitted for the practical exam held on:

Internal Examiner External Examiner

Date: Date:

VISION OF DEPARTMENT:

To be a leading force in leveraging cutting-edge technology to drive students with innovation, efficiency, and strategic growth.

MISSION OF DEPARTMENT:

M1: Academic Excellence: To provide high-quality education that equips students with the theoretical knowledge and practical skills necessary to excel in their careers and adapt to the evolving technological landscape.

M2: Innovative Research: To foster a research-oriented environment that encourages innovation, interdisciplinary collaboration, and the pursuit of cutting-edge solutions to complex problems in technology and related fields.

M3: Ethical and Inclusive Education: To promote ethical practices, diversity, and inclusion within the IT field, preparing students to be responsible and culturally aware professionals.

PROGRAM EDUCATIONAL OBJECTIVES:

PEO1: Domain Knowledge: To produce graduates who have strong foundation of knowledge and skills in the field of Information Technology.

PEO2: Employability Skills and Research: To produce graduates who are employable in industries/ public sector/ research organizations or work as an entrepreneur.

PEO3: Ethics and Values: To develop leadership skills and ethically collaborate with society to tackle real-world challenges.

PROGRAM OUTCOMES:

Engineering Graduates will be able to:

PO1: Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.

PO2: Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)

PO3: Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with

consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)

PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).

PO5: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)

PO6: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).

PO7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)

PO8: Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9: Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences

PO10: Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11: Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.(WK8)

PROGRAMME SPECIFIC OUTCOME

PSO 1: System Design and Development: Demonstrate the ability to apply essential IT and computing principles to develop robust and efficient software solutions.

PSO 2: Data Communication and Networking: Analyze and evaluate system resources to ensure scalability, and optimal performance in distributed and networked environments.

PSO 3: Computational and Data Intelligence: Design and deployment of intelligent systems to uncover hidden patterns in complex datasets that make data-driven decisions

COURSE OUTCOME

CO1	Demonstrate their ability to implement fundamental cryptographic algorithms.
CO2	Implement symmetric and asymmetric encryption algorithms and apply them to secure messages in a network environment.
CO3	Implement and analyze message authentication codes and digital signatures to ensure the authenticity and integrity of messages.
CO4	Set up and configure honey pots using KF Sensor, deploying the mina network environment to attract and analyze traffic
CO5	Configure and use Net Stumbler to discover and analyze available wireless networks, including those using WEP and WPA encryption.

MAPPING OF COURSE OUTCOMES WITH PROGRAMME OUTCOMES

CO	Programme Outcomes (PO) & Programme Specific Outcomes (PSO)													
	PO1	**PO2**	**PO3**	**PO4**	**PO5**	**PO6**	**PO7**	**PO8**	**PO9**	**PO10**	**PO11**	**PSO1**	**PSO2**	**PSO3**
CO1	3	3	1	2	3			1			2	3	2	1
CO2	3	2	1	1	3			1			1	3	2	1
CO3	3	2	3	2	3			1		1	2	3	2	1
CO4	3	2	3	2	3	1		1		1	2	3	2	1
CO5	3	2	3	2	3	1		1		1	2	3	2	1

Lab Assessment Rubrics

Criteria	Excellent	Good	Satisfactory	Unsatisfactory
Record (15 marks)				
Program Execution (5)	Program runs correctly with expected output and clear logic.. (5)	Program runs with minor errors or partial output; logic mostly correct. (3-4)	Major logic/syntax errors; program does not compile or run meaningfully. (1-2)	No attempt made or program is irrelevant/incomplete. (0)
Screenshots and Output Evidence (5)	Clear, complete screenshots for all steps, properly labeled (5)	Few steps missed or minor labeling issues (3-4)	Incomplete or unclear screenshots (1-2)	Missing or irrelevant screenshots (0)
Presentation & Timely Submission (5)	Well-formatted, neat, easy to read, and submitted on or before the deadline. (5)	Mostly neat with minor formatting issues, submitted late by 1 day. (3-4)	Average formatting with some clutter or inconsistencies, submitted late by 2 days. (1-2)	Poorly presented and disorganized, and not submitted. (0)
Viva (10 marks)				
Knowledge & Understanding (6)	Comprehensive and accurate understanding demonstrated during viva. (5-6)	Good understanding, with some minor gaps or inaccuracies. (3-4)	Adequate understanding, with noticeable gaps or inaccuracies. (1-2)	Poor understanding, with critical gaps or inaccuracies. (0)
Communication Skills (4)	Clear, concise, and effective communication during viva. (4)	Mostly clear communication with minor issues. (3)	Communication is understandable but may lack clarity at times. (1-2)	Communication is unclear, disorganized, or incoherent. (0)

EXP.NO	DATE	NAME OF THE PROGRAM	PAGE NO.
1(A)	10/07/26	Create a Database in excel and convert it into CSV files	1
1(B)	10/07/26	Upload the CSV file and Analyse datasets using data mining tool.	4
1(C)	31/07/26	Explore the data types and variables and analyze the features	8
1(D)	31/07/26	Normalize the datasets using preprocessing techniques using WEKA tool	13
2	31/07/26	Install and explore the features of NumPy, SciPy, Jupyter, Statsmodels and Pandas packages.	24
3	06/08/26	Working with Numpy Arrays	30
4	03/08/26	Working with Pandas data frames	30
5	10/09/26	Descriptive Analysis: Reading Data from different files and web sources using Python	36
6	24/09/26	Working with Univariate and Bivariate Analysis	42

Ex.No. 1a	Create a Database in excel and convert it into CSV files
DATE: 10.07.2026	

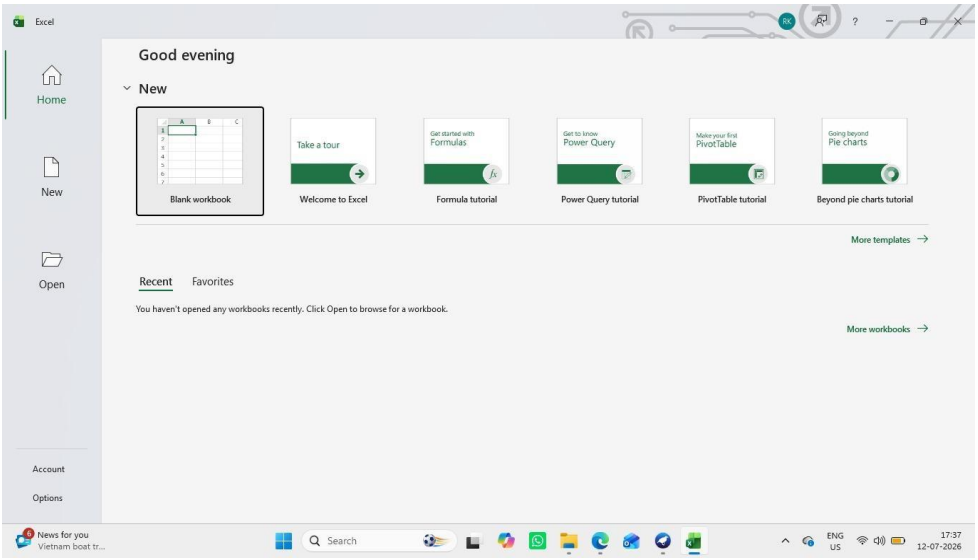
Objective:

To create a database in Microsoft Excel and convert it into a CSV file for further analysis.

Procedure:

1. Open a Microsoft excel.

2. Create a New workbook. Click on FILE → NEW → BLANK WORKBOOK



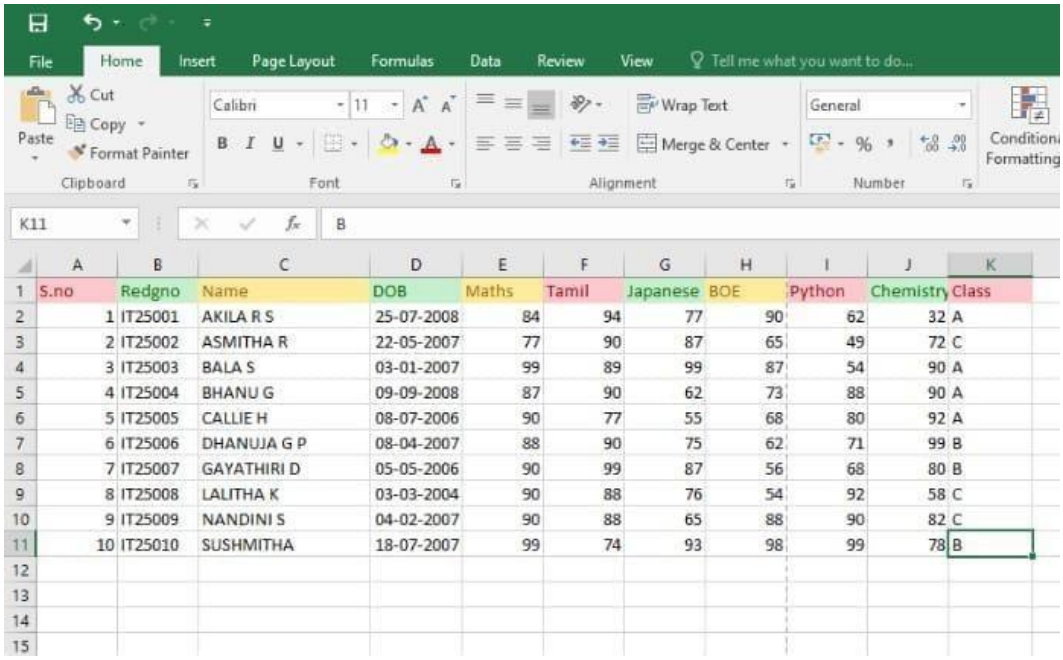
3. Enter the data in the worksheet.

S.No	Redgno	Name	DOB	Math	Tamil	Jap	Boe	Python	Chem	Class
1	IT25001	AKILA R S	25-07-2008	84	94	77	90	62	32	A
2	IT25002	ASMITHA R	22-05-2007	77	90	87	65	49	72	C
3	IT25003	BALA S	03-01-2007	99	89	99	87	54	90	A
4	IT25004	BHANU G	09-09-2008	87	90	62	73	88	90	A
5	IT25005	CALLIE	08-07-2006	90	77	55	68	80	92	A
6	IT25006	DHANUJA G P	08-04-2007	88	90	75	62	71	99	B

7	IT25007	GAYATHIRI D	05-05-2006	90	99	87	56	68	80	B
8	IT25008	LALITHA K	03-03-2004	90	88	76	54	92	58	C
9	IT25009	NANDINI S	04-02-2007	90	88	65	88	90	82	C
10	IT25010	SUSHMITHA	18-07-2007	99	74	93	98	99	78	B

4. Save the excel file

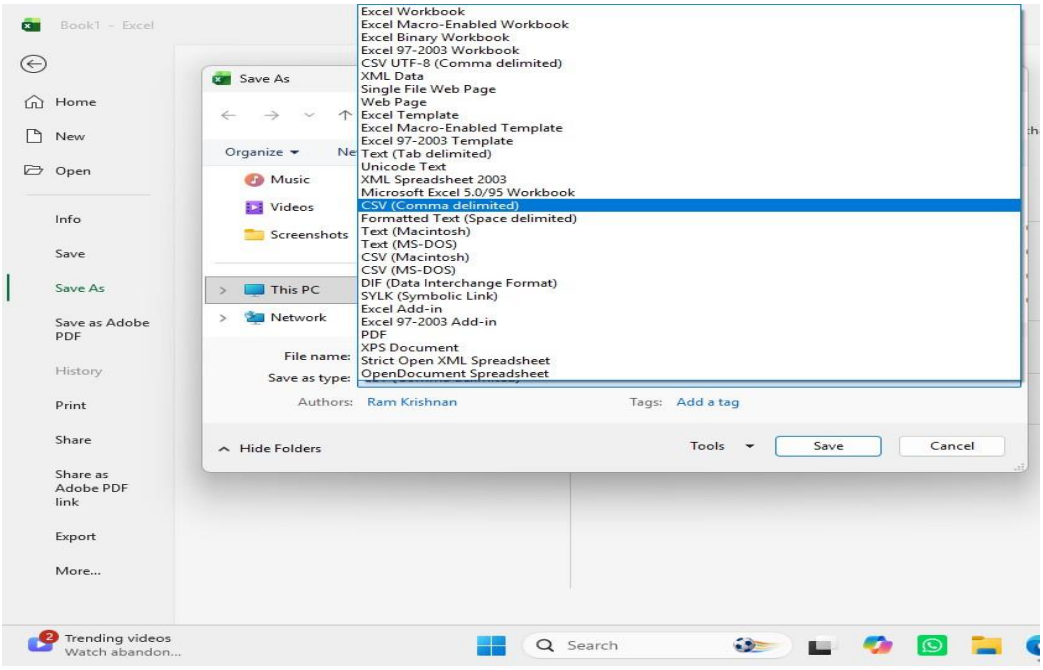
- Click on "File" > "Save As".
- Choose a location to save the file.



	A	B	C	D	E	F	G	H	I	J	K
1	S.no	Redgno	Name	DOB	Maths	Tamil	Japanese	BOE	Python	Chemistry	Class
2	1	IT25001	AKILA R S	25-07-2008	84	94	77	90	62	32	A
3	2	IT25002	ASMITHA R	22-05-2007	77	90	87	65	49	72	C
4	3	IT25003	BALA S	03-01-2007	99	89	99	87	54	90	A
5	4	IT25004	BHANU G	09-09-2008	87	90	62	73	88	90	A
6	5	IT25005	CALLIE H	08-07-2006	90	77	55	68	80	92	A
7	6	IT25006	DHANUJA G P	08-04-2007	88	90	75	62	71	99	B
8	7	IT25007	GAYATHIRI D	05-05-2006	90	99	87	56	68	80	B
9	8	IT25008	LALITHA K	03-03-2004	90	88	76	54	92	58	C
10	9	IT25009	NANDINI S	04-02-2007	90	88	65	88	90	82	C
11	10	IT25010	SUSHMITHA	18-07-2007	99	74	93	98	99	78	B
12											
13											
14											
15											

5. Enter a name for your file and click "Save".5. Convert the Excel File to CSV

- Save as CSV
- Click on "File" > "Save As".
- Choose the location where you want to save the CSV file.
- In the "Save as type" dropdown, select "CSV (Comma delimited) (*.csv)".
- Enter a name for your CSV file and click "Save".



6. Confirm the CSV file creation

- Confirm the CSV Format
- A warning might pop up stating that some features in your workbook might be lost if you save it as CSV. Click "Yes" to continue.

Conclusion:

The process of creating a database in Excel and converting it into a CSV file was completed successfully.

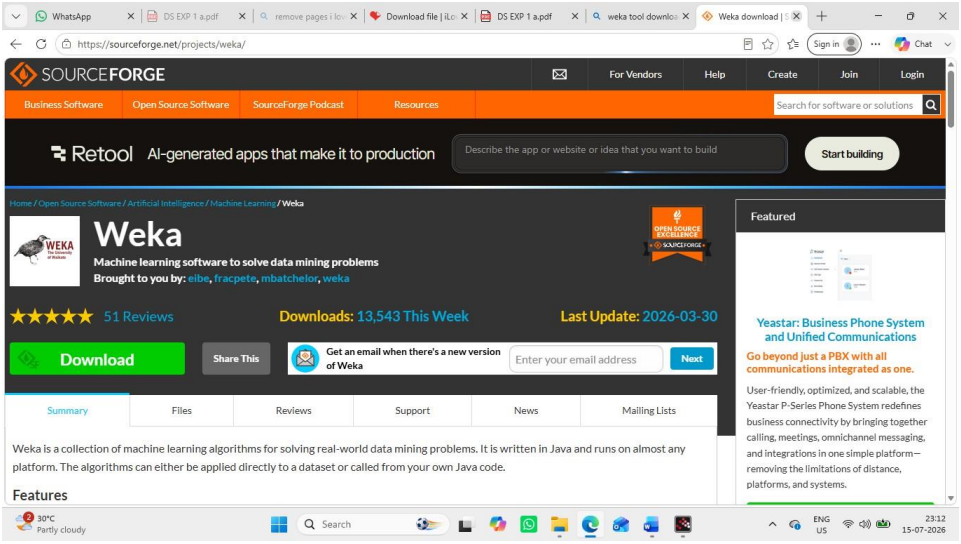
Ex.No. 1b	Upload the CSV the file and Analyse datasets using data mining tool
DATE:10.07.2026	

Objective:

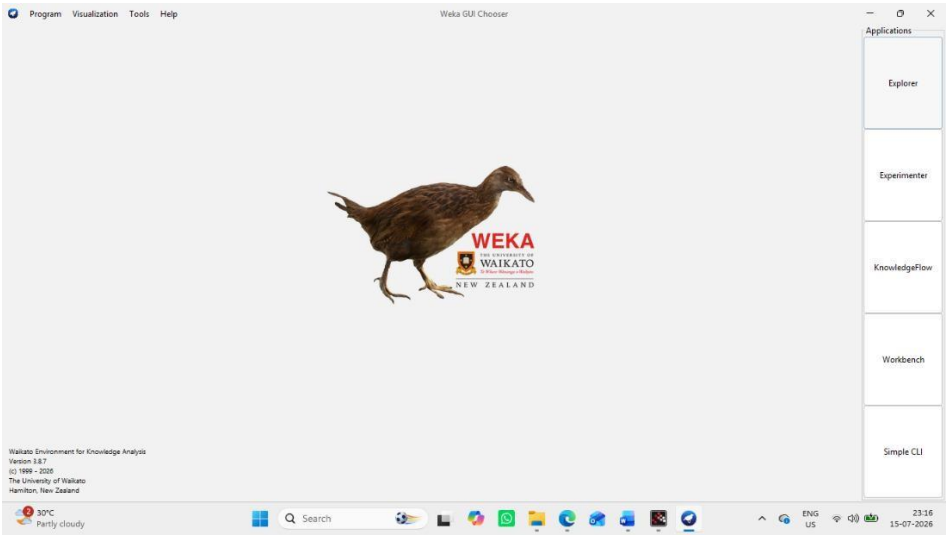
To upload the CSV the file and analyse datasets using data mining tool.

Procedure:

1. Install data mining tool (weka tool).

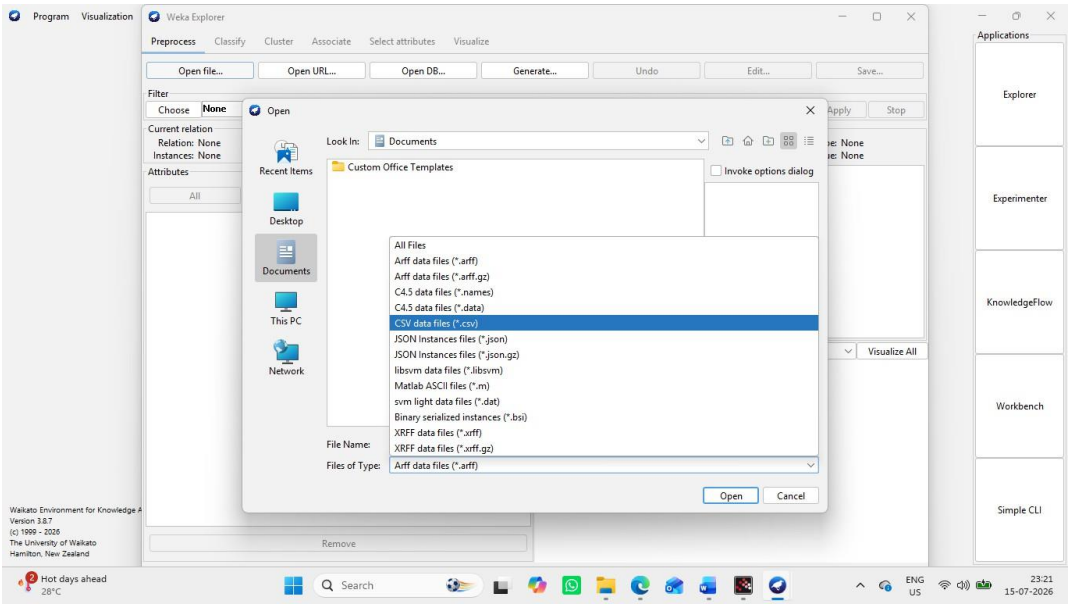


2. Click Explore to go inside the weka tool for analysis.



3. Click open file to upload the CSV file into the weka tool for analysis.

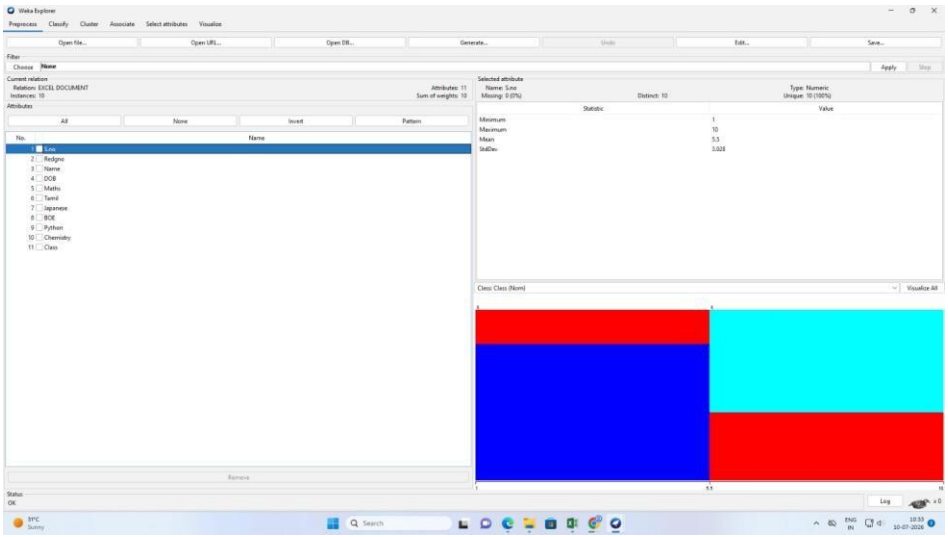
- Search the CSV file after choosing the file type as CSV.
- Open that file for analysis.
- The only allowed file in data mining is CSV file.

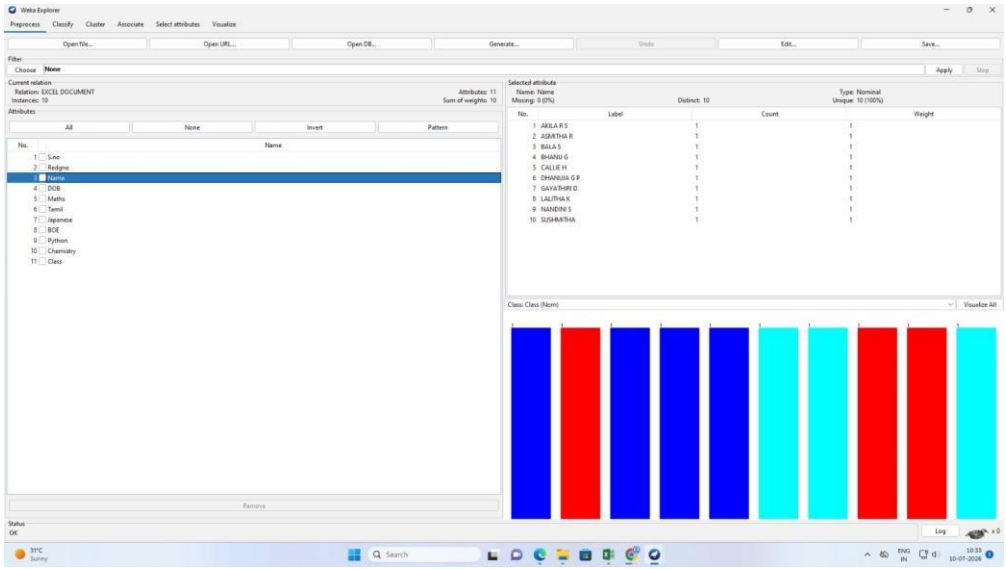


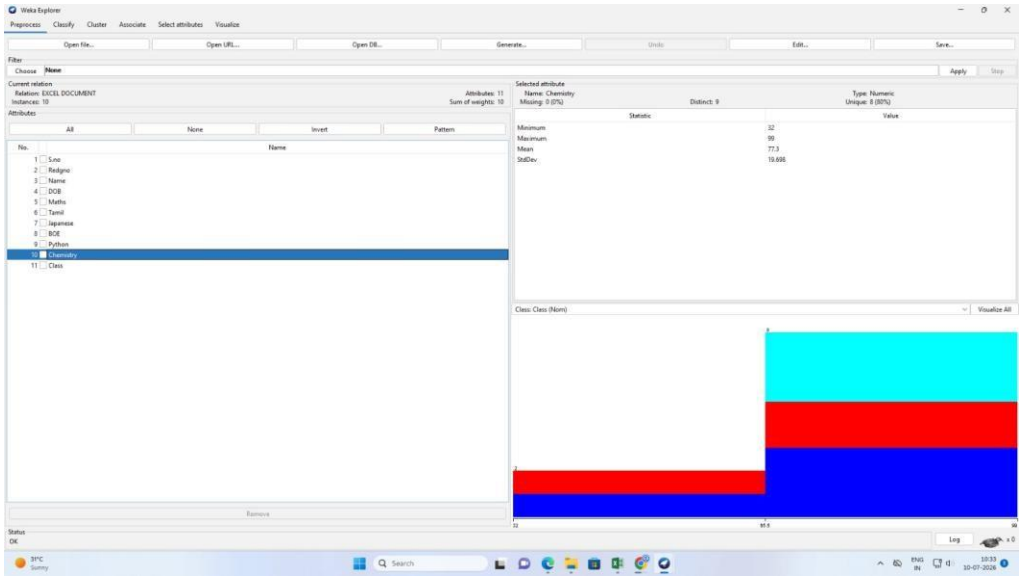
4. Load the CSV file into the mining tool for the analysing Process.

5. Analyse each data in that selected CSV file.

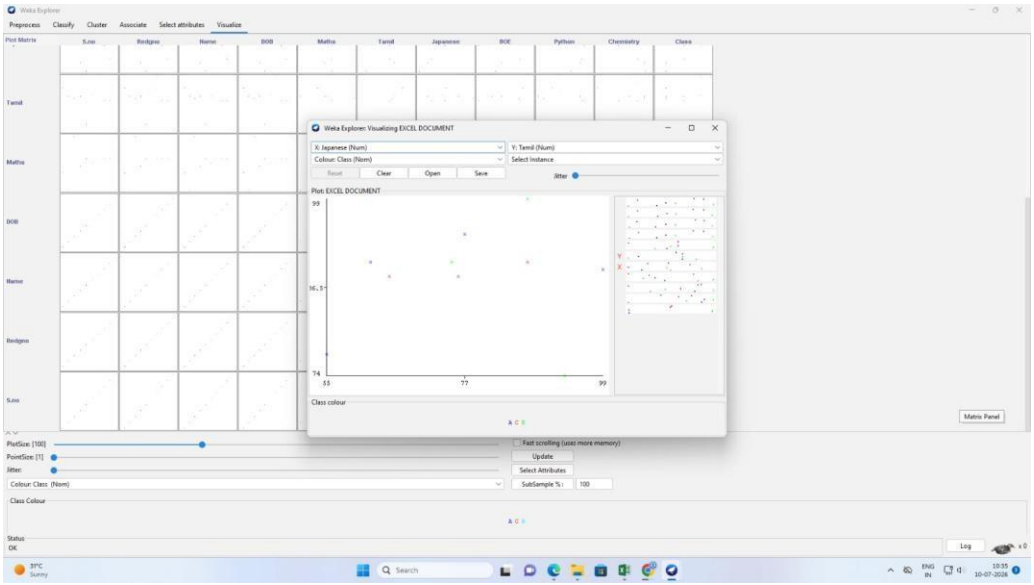
- Click each data to analyse it.
- The three data analysis given below.
- This represents the variation of the data in the csv file







6. Click the visualize.
   - For viewing the Graphical representation of the datas.
   - The Graph Represents the analaysed datas using weka mining tool..



6.

Conclusion:

The process of analysing the datasets using data mining tool (weak tool) of a CSV file was completed successfully.

Ex. No.1c Explore the data types and variables and analyze the features

DATE: 31/07/2026

Aim:

To explore the data types and variables and analyze the features using Weka data mining tool.

Procedure:

1. Open Weka

- Open Weka
- Launch the Weka application on your computer.

2. Load the CSV File

- Load the CSV File
- In the Weka GUI, click on "Explorer".
- Click on the "Open file..." button.
- Navigate to the location where you saved the CSV file, select it, and click "Open".

3. Explore Data Types and Variables

- Preprocess Tab
- Once the file is loaded, you will be in the "Preprocess" tab.
- The "Attributes" panel will list all the variables (columns) in your dataset.

Data Types

- Each attribute will have a data type such as numeric, nominal (categorical), or string.
- Click on each attribute to see its details, including the data type and distribution of values.

4. Analyze Features

a. Summary Statistics

- For numeric attributes, you can view summary statistics like mean, min, max, and standard deviation.

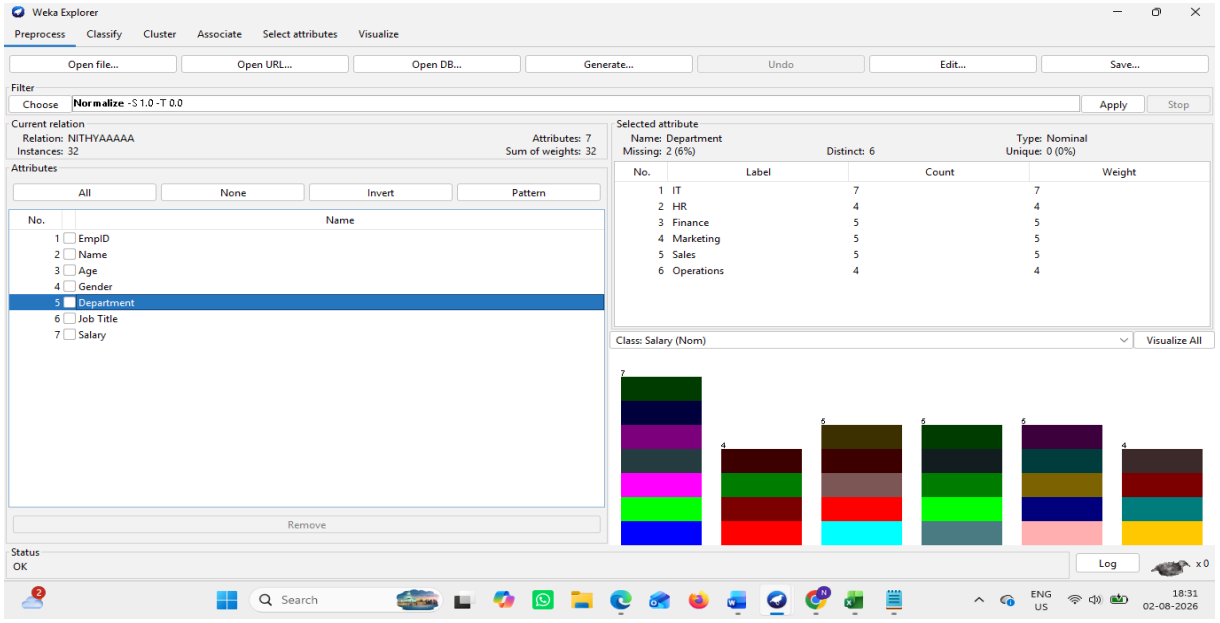
For nominal attributes, you can see the frequency of each category

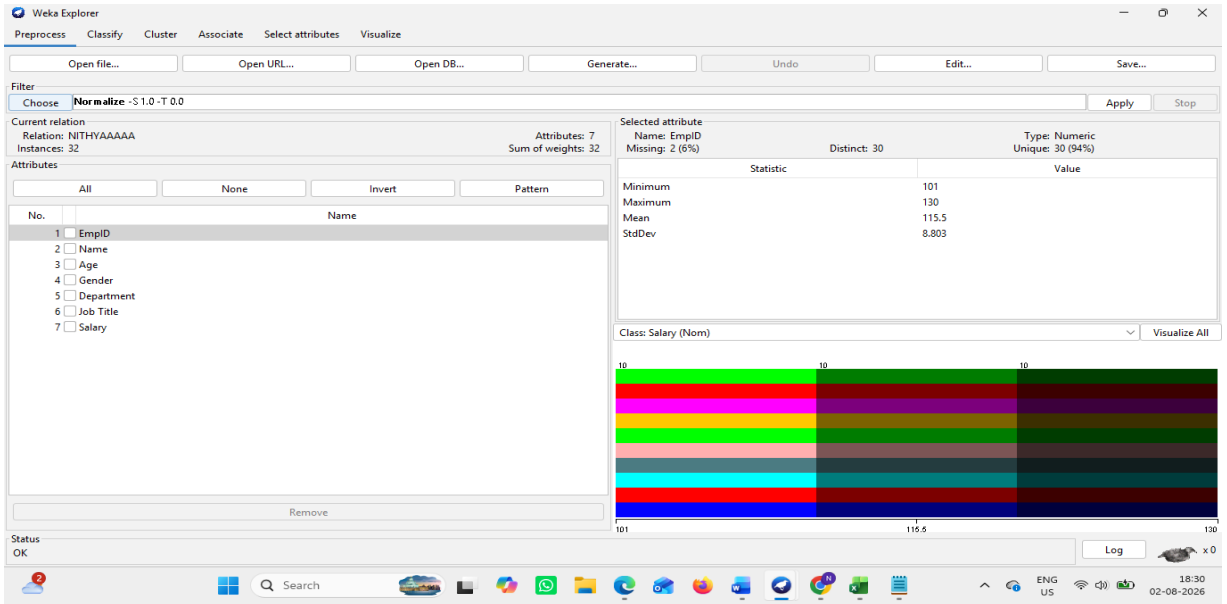
b.Tree analysis using J48

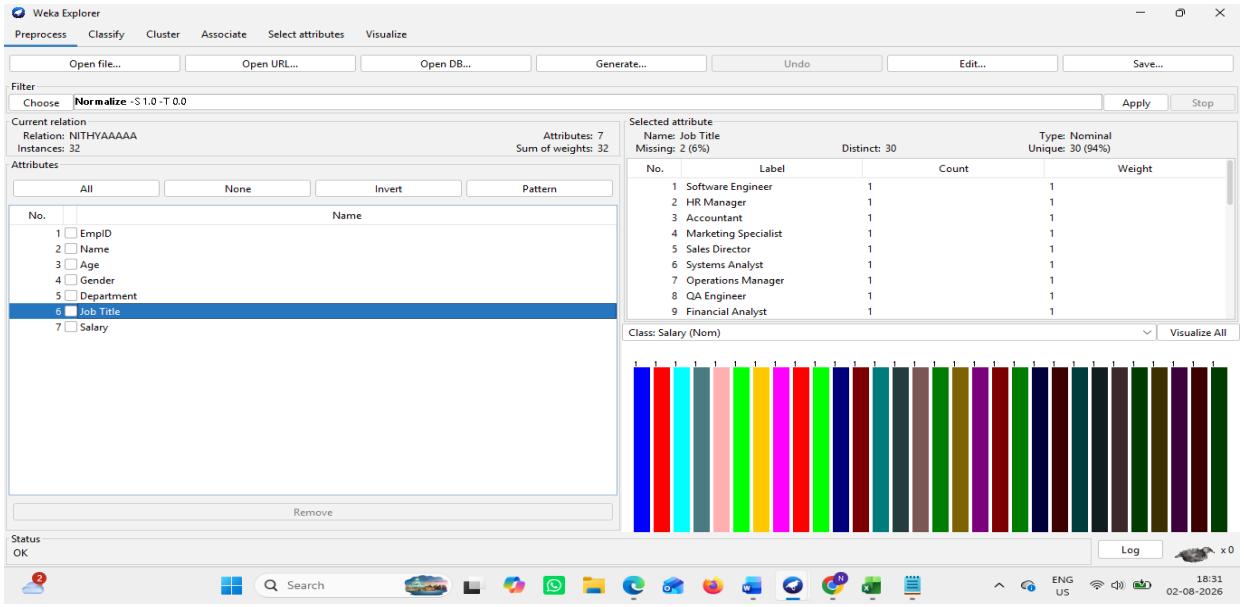
- Click on the "Class" tab.
- Use the "J-48" method to evaluate the relationship between features.
- Click "Start" to run the analysis.

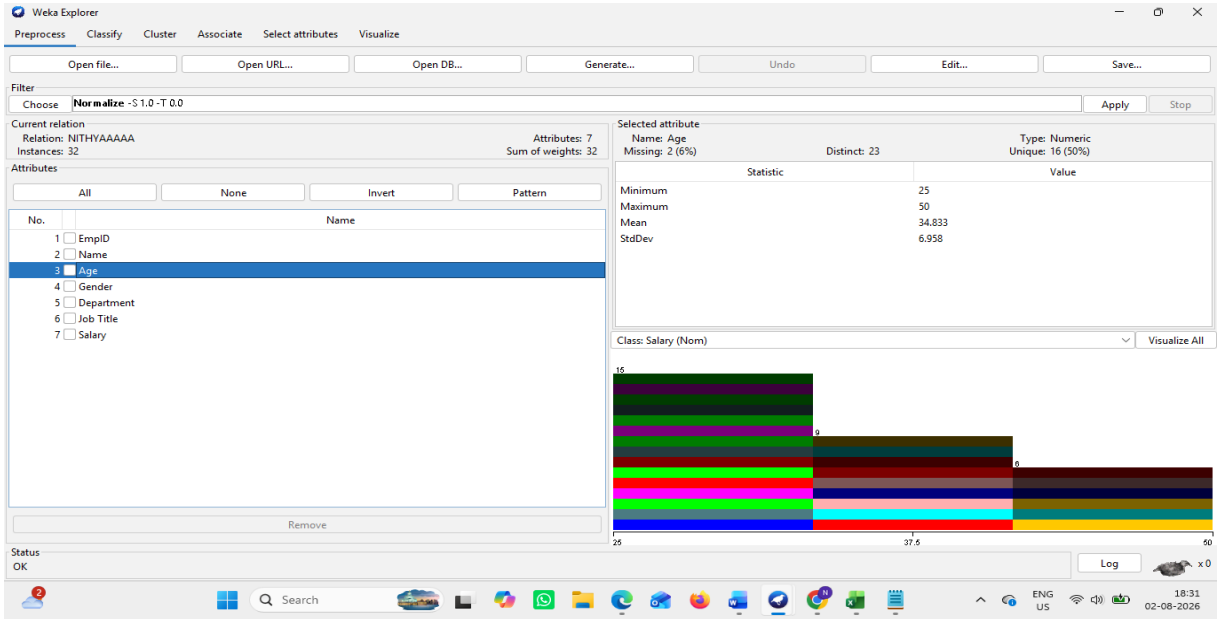
visual Exploration

- Click on the "Visualize" tab to create scatter plots, histograms, or other visualizations.
- Select different pairs of attributes to visualize relationships and distributions.


Weka Explorer
Preprocess
Classify
Cluster
Associate
Select attributes
Visualize
Open file...
Open URL...
Open DB...
Generate...
Undo
Edit...
Save...
Filter
Choose
Normalize -S 1.0 -T 0.0
Apply
Stop
Current relation
Relation: NITHYAAAAA
Instances: 32
Attributes: 7
Sum of weights: 32
Attributes
All
None
Invert
Pattern
No.
Name
1 EmpID
2 Name
3 Age
4 Gender
5 Department
6 Job Title
7 Salary
Remove
Selected attribute
Name: Department
Missing: 2 (6%)
Distinct: 6
Type: Nominal
Unique: 0 (0%)
No.
Label
Count
Weight
1 IT 7 7
2 HR 4 4
3 Finance 5 5
4 Marketing 5 5
5 Sales 5 5
6 Operations 4 4
Class: Salary (Nom)
Visualize All
Status
OK
Log
x 0
Search
ENG
US
18:31
02-08-2026


Weka Explorer
Preprocess
Classify
Cluster
Associate
Select attributes
Visualize
Open file...
Open URL...
Open DB...
Generate...
Undo
Edit...
Save...
Filter
Choose
Normalize -S 1.0 -T 0.0
Apply
Stop
Current relation
Relation: NITHYAAAAA
Instances: 32
Attributes: 7
Sum of weights: 32
Attributes
All
None
Invert
Pattern
No.
Name
1 EmpID
2 Name
3 Age
4 Gender
5 Department
6 Job Title
7 Salary
Remove
Selected attribute
Name: EmpID
Missing: 2 (6%)
Distinct: 30
Type: Numeric
Unique: 30 (94%)
Statistic
Value
Minimum 101
Maximum 130
Mean 115.5
StdDev 8.803
Class: Salary (Nom)
Visualize All
101
115.5
130
Status
OK
Log
x 0
Search
ENG
US
18:30
02-08-2026


Weka Explorer
Preprocess
Classify
Cluster
Associate
Select attributes
Visualize
Open file...
Open URL...
Open DB...
Generate...
Undo
Edit...
Save...
Filter
Choose
Normalize -S 1.0 -T 0.0
Apply
Stop
Current relation
Relation: NITHYAAAAA
Instances: 32
Attributes: 7
Sum of weights: 32
Attributes
All
None
Invert
Pattern
No.
Name
1 EmpID
2 Name
3 Age
4 Gender
5 Department
6 Job Title
7 Salary
Remove
Selected attribute
Name: Job Title
Missing: 2 (6%)
Distinct: 30
Type: Nominal
Unique: 30 (94%)
No.
Label
Count
Weight
1 Software Engineer 1 1
2 HR Manager 1 1
3 Accountant 1 1
4 Marketing Specialist 1 1
5 Sales Director 1 1
6 Systems Analyst 1 1
7 Operations Manager 1 1
8 QA Engineer 1 1
9 Financial Analyst 1 1
Class: Salary (Nom)
Visualize All
Status
OK
Log
x 0
Search
ENG
US
18:31
02-08-2026


Weka Explorer
Preprocess
Classify
Cluster
Associate
Select attributes
Visualize
Open file...
Open URL...
Open DB...
Generate...
Undo
Edit...
Save...
Filter
Choose
Normalize -S 1.0 -T 0.0
Apply
Stop
Current relation
Relation: NITHYAAAAA
Instances: 32
Attributes: 7
Sum of weights: 32
Attributes
All
None
Invert
Pattern
No.
Name
1 EmpID
2 Name
3 Age
4 Gender
5 Department
6 Job Title
7 Salary
Remove
Selected attribute
Name: Age
Missing: 2 (6%)
Distinct: 23
Type: Numeric
Unique: 16 (50%)
Statistic
Value
Minimum 25
Maximum 50
Mean 34.833
StdDev 6.958
Class: Salary (Nom)
Visualize All
15
9
6
25
37.5
50
Status
OK
Log
x 0
Search
ENG
US
18:31
02-08-2026

Selected attribute

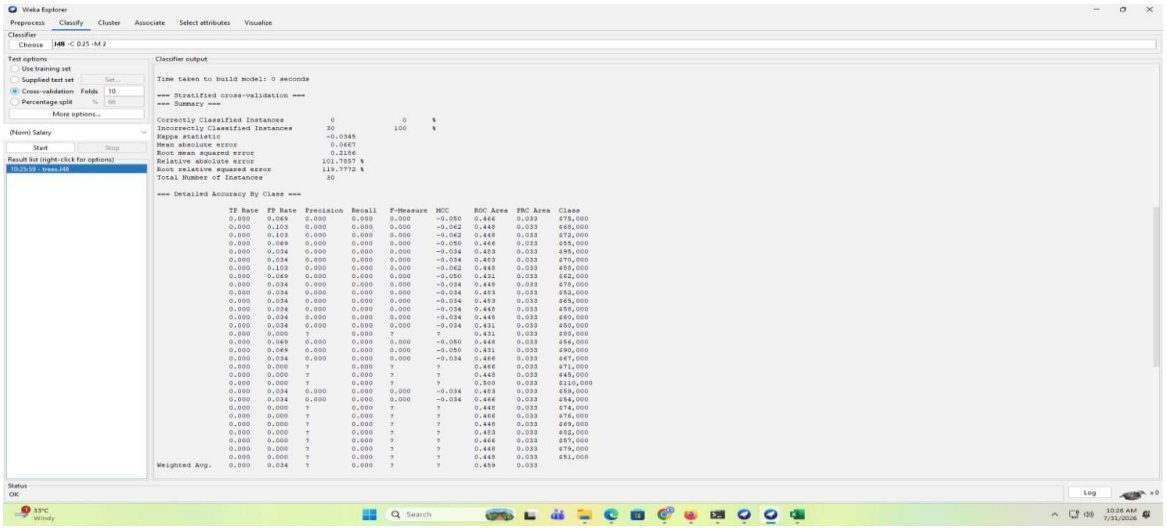
Name: EmpID | Type: Numeric
Missing: 2 (6%) | Distinct: 30 | Unique: 30 (94%)

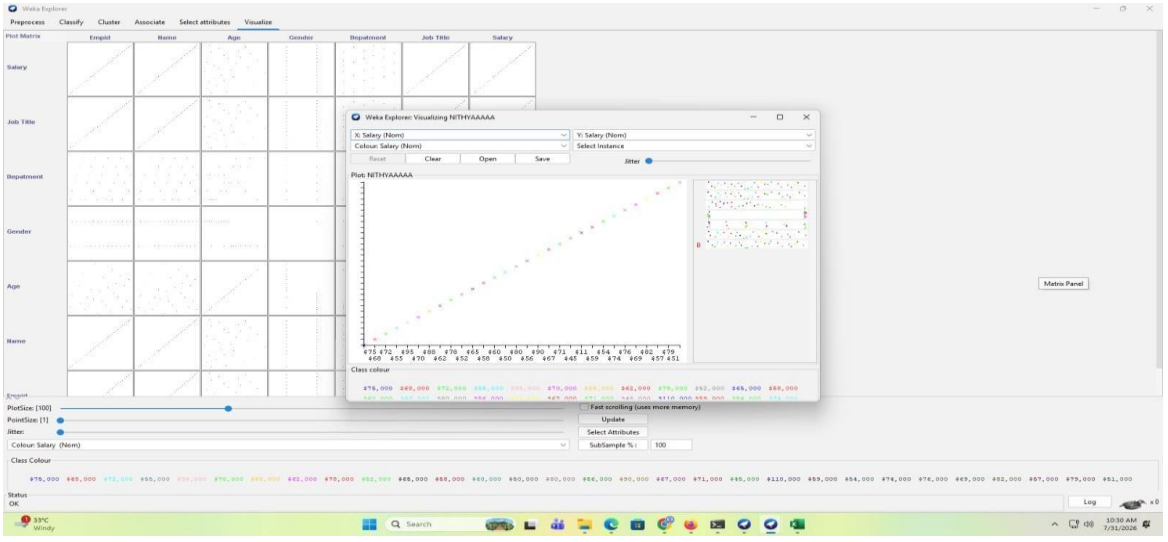
Statistic	Value
Minimum	101
Maximum	130
Mean	115.5
StdDev	8.803

Selected attribute

Name: Name | Type: Nominal
Missing: 2 (6%) | Distinct: 30 | Unique: 30 (94%)

No.	Label	Count	Weight
1	John Smith	1	1
2	Jane Doe	1	1
3	Robert Johnson	1	1
4	Emily Davis	1	1
5	Michael Brown	1	1
6	Sarah Wilson	1	1
7	David Lee	1	1
8	Anna Taylor	1	1
9	James Anderson	1	1





Result:

Thus the dataset was thoroughly explored for data types and variables. Summary statistics and frequency distributions provided an understanding of each attribute's characteristics

Ex.No.1d: Normalize the datasets using preprocessing techniques using WEKA tool

DATE: 31/07/2026

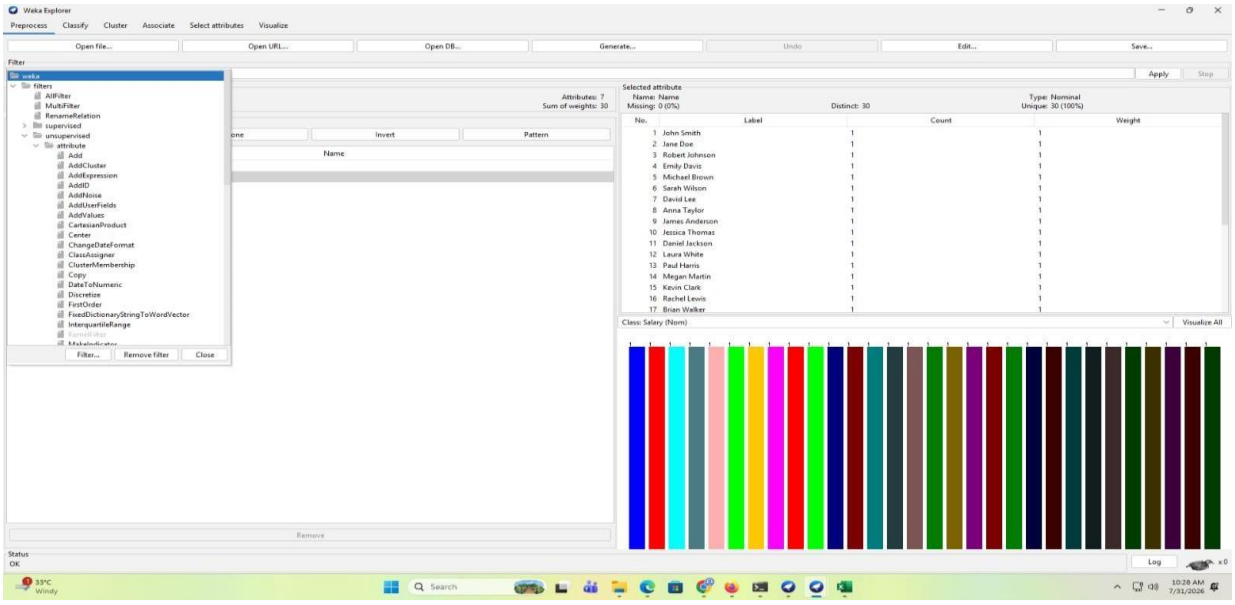
Aim:

To Normalize the datasets using preprocessing techniques using WEKA tool

Procedure:

1. Open Weka
   - Open Weka
   - Launch the Weka application on your computer.

2. Load the CSV File
   - Load the CSV File
   - In the Weka GUI, click on "Explorer".
   - Click on the "Open file..." button.
   - Navigate to the location where you saved the CSV file, select it, and click "Open".

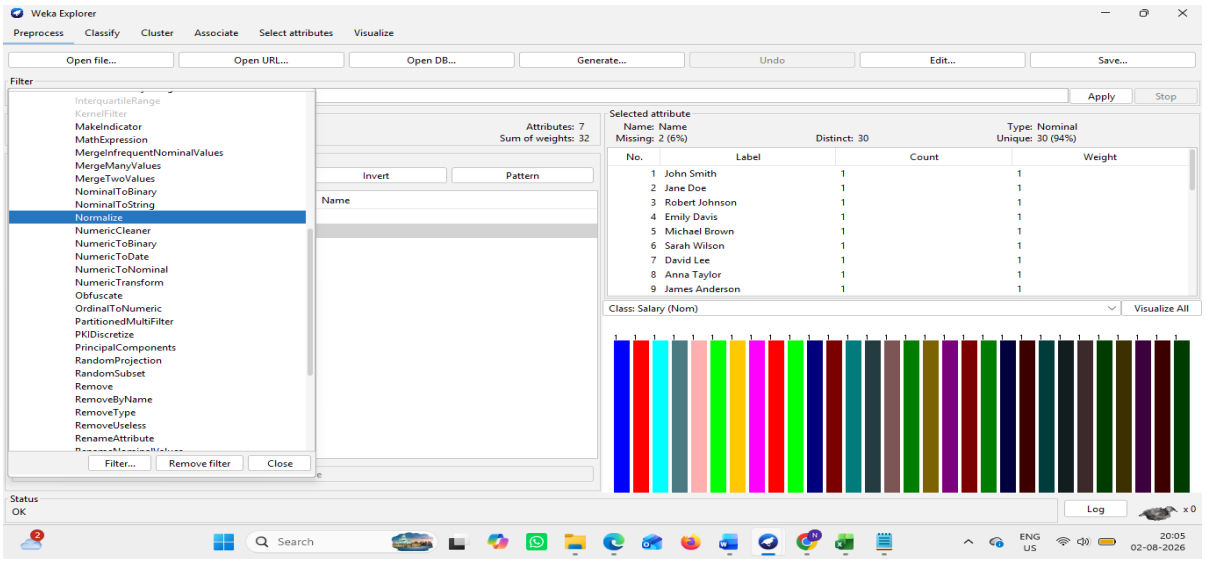


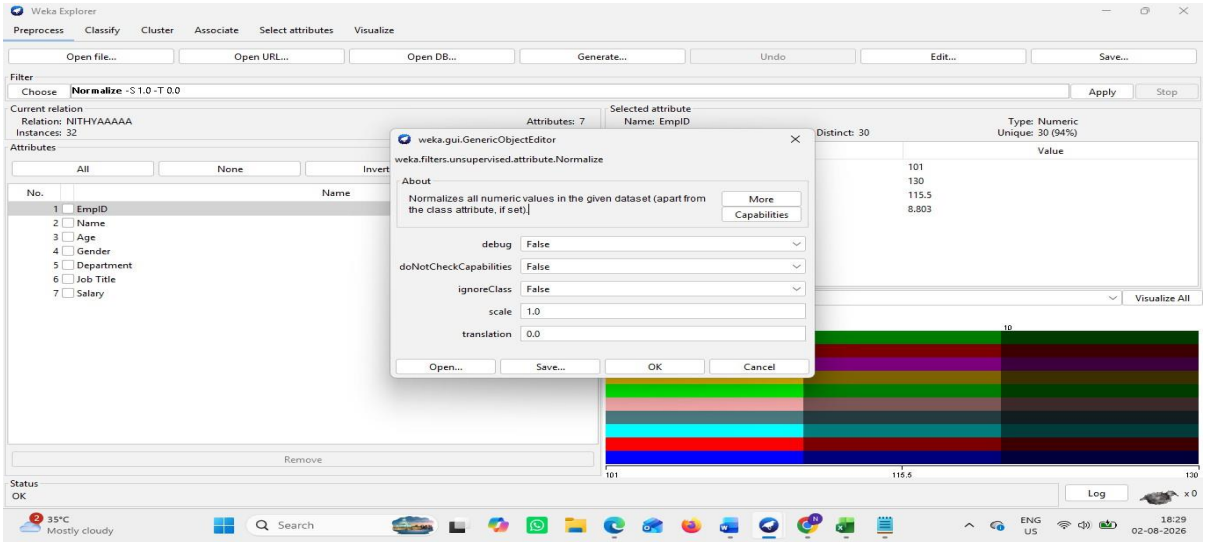
3. Apply Normalization
   - Go to the Preprocess Tab
   - Ensure that your dataset is loaded and visible in the "Preprocess" tab.

- Click on the "Choose" button under the "Filter" section.
- Navigate to "unsupervised" > "attribute" > "Normalize".

4. Configure Normalize Filter

- Once the Normalize filter is selected, click on the "Normalize" filter to open its configuration settings.
- Set the desired range for normalization (e.g., default [0, 1]).



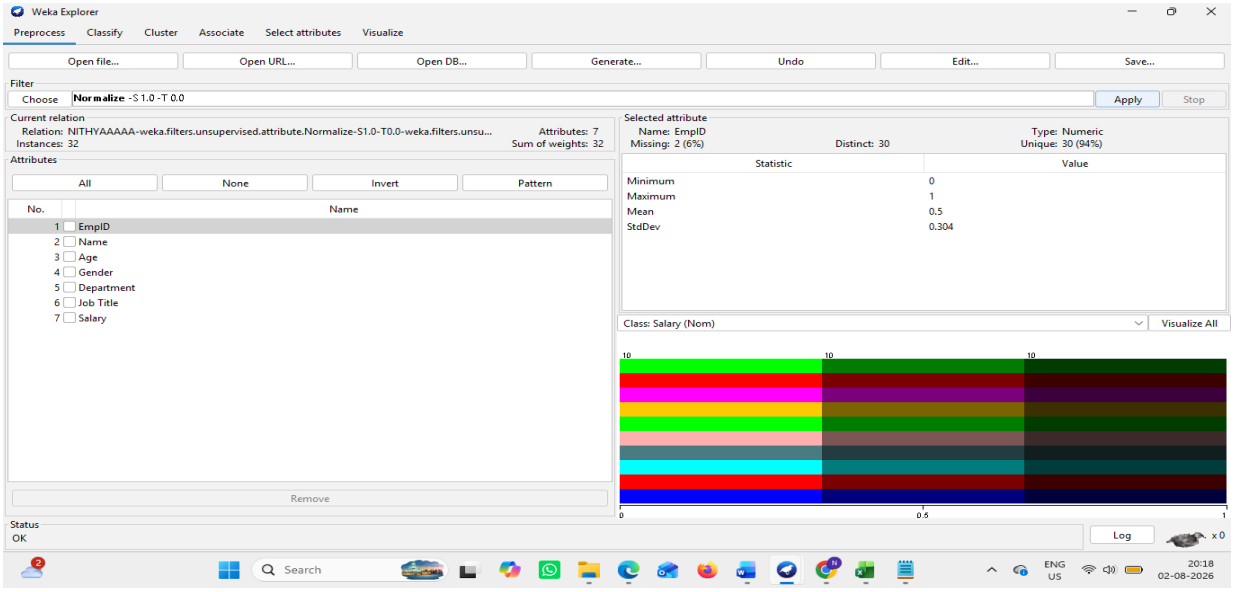


5. Apply the Filter

- Click the "Apply" button to normalize the dataset.
- The dataset should now reflect the normalized values.

6. Save the Normalized Dataset

- Save the Normalized Data
- Click on "Save" to save the normalized dataset.
- Choose the location and name for the new CSV file, and ensure the file type is set to CSV



Result:

By using Weka's Normalize filter, the numerical attributes in the dataset were scaled to a standard range, ensuring better performance for subsequent data mining and analysis processes.

EX.2 Install and explore the features of NumPy, SciPy, Jupyter, Statsmodels and Pandas packages.

DATE:31/07/2026

Objective:

To install and explore the features of NumPy, SciPy, Jupyter, Statsmodels, and Pandas packages.

PROCEDURE

Step 1: Download and Install Python with IDLE

Download Python:

Go to the official Python website: https://www.python.org/.

Click on the "Downloads" tab.

Download the latest version of Python for your operating system.

Install Python:

Run the downloaded installer.

During installation, make sure to check the boxthat says "Add Python to PATH" before clicking "Install Now".

Follow the prompts to complete the installation.

Step 2: Install Necessary Packages

Open Command Prompt or Terminal:

For Windows: Press Win + R, type cmd, and press Enter.

In the command prompt or terminal, type the following commands to install the required packages:

pip install numpy scipy statsmodels pandas matplotlib

Press Enter after eachcommand and wait for the installation to complete.

Step 3: Open Python IDLE

Open Python IDLE:

For Windows: Find Python IDLE in the start menu or search for "IDLE".

Step 4: Write and Runthe Python Script

Create a New Script:

In Python IDLE, go to File > New File to opena new script window.

Step 5: Explore NumPy Features:

Import the NumPy library and create NumPyarrays and perform basic operations like addition, mean calculation, and standard deviation.

Step 6: Explore Pandas Features:

Import the Pandas library and create a DataFrame with sample data and perform DataFrame operations such as calculating mean, median, and mode.

Step 7:Explore SciPy Features:

Import the SciPystats module and performa t-test on two sets ofdata and print the t-statistic and p-value to interpret the results of the test.

Step 8:Explore Statsmodels Features:

Import the Statsmodels library and fit a simple linear regression model and use sample data to predict adependent variable based on independent variables.

Step 9:Plotting with Matplotlib in Jupyter Notebook:

Import the Matplotlib library to create plots and plot a scatter plot to visualize the relationship between two variables.

Program:

```
# Import necessary libraries
import numpy as np

import pandas as pd

import scipy.stats as stats

import statsmodels.api as sm

import matplotlib.pyplot as plt

# 1. NumPy: Numerical Python

# Create a NumPyarrayand perform basic operations

a = np.array([1, 2, 3, 4, 5])

b= np.array([10, 20, 30, 40, 50])

# Basic operations

sum_ab = a + b

mean_a = np.mean(a)

std_b = np.std(b)

print("NumPy Operations:")

print("Sumofarrays a and b:", sum_ab)

print("Mean of array a:", mean_a)

print("Standard deviation of array b:", std_b)

print("\n")

# 2. Pandas: Data manipulation and analysis

# Create a Pandas DataFrame

data = {

    'Age': [23, 25, 28, 22, 30],

    'Weight': [70, 65, 78, 55, 72],

    'Height': [165, 170, 175, 160, 168]

}

df = pd.DataFrame(data)

# DataFrame operations
```

```
mean_age = df['Age'].mean()
median_weight = df['Weight'].median()
mode_height = stats.mode(df['Height'], keepdims=True)[0][0]
print("Pandas Operations:")
print("Mean age:", mean_age)
print("Medianweight:", median_weight)
print("Mode height:", mode_height)
print("\n")
# 3. SciPy: Scientific and technical computing
# Perform a t-test ontwo sets of data
t_stat, p_value = stats.ttest_ind(a, b)
print("SciPy Operations:")
print("T-statistic:", t_stat)
print("P-value:", p_value)
print("\n")
```

```
# 4. Statsmodels: Statistical models
# Fit a simple linear regression model
X = sm.add_constant(df[['Age', 'Weight']]) # Add constant term
y = df['Height']
model= sm.OLS(y, X)
results = model.fit()
print("Statsmodels OLS Regression Results:")
print(results.summary())
print("\n")
# 5. Matplotlib: Plotting graphs
```

```
# Plot a graph
plt.scatter(df['Age'], df['Height'], color='blue')
plt.xlabel('Age')
plt.ylabel('Height')
plt.title('Age vs. Height')
plt.show()
```

```
NumPy Operations:
Sum of arrays a and b: [11 22 33 44 55]
Mean of array a: 3.0
Standard deviation of array b: 14.142135623730951


Pandas Operations:
Mean age: 25.6
Median weight: 70.0
Mode height: 160


SciPy Operations:
T-statistic: -3.7994267415435763
P-value: 0.005240940516978861
```

Statsmodels OLS Regression Results:

OLS Regression Results

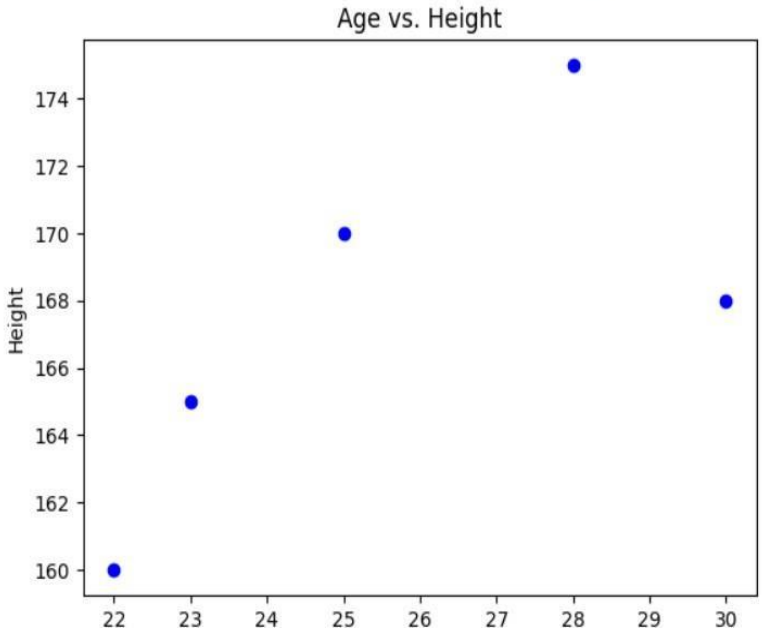
Dep. Variable:	Height	R-squared:	0.717
Model:	OLS	Adj. R-squared:	0.434
Method:	Least Squares	F-statistic:	2.535
Date:	Mon, 17 Aug 2026	Prob (F-statistic):	0.283
Time:	14:26:15	Log-Likelihood:	-11.989
No. Observations:	5	AIC:	29.98
Df Residuals:	2	BIC:	28.81
Df Model:	2		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
const	128.8596	17.631	7.308	0.018	52.997	204.722
Age	0.2859	0.919	0.311	0.785	-3.670	4.241
Weight	0.4621	0.358	1.291	0.326	-1.078	2.003

Omnibus:	nan	Durbin-Watson:	1.709
Prob(Omnibus):	nan	Jarque-Bera (JB):	0.547
Skew:	0.315	Prob(JB):	0.761
Kurtosis:	1.507	Cond. No.	685.

```
Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.


/usr/local/lib/python3.12/dist-packages/statsmodels/stats/stattools.py:74: ValueWarning: omni_normtest is not valid with less t
  warn("omni_normtest is not valid with less than 8 observations; %i "
```



Conclusion:

We explore the features of NumPy, SciPy, Pandas, Statsmodels, and Matplotlib in Python IDLE successfully.

Ex.No. 3	Working with NumPy arrays
DATE: 06.08.26	

a. Create an array using “np” library with different methods like arrange(), random.rand(), empty()

b. Create a Numpy array filled with zeros and ones.

c. Create Student Database with five columns and remove the columns in Numpy arrary that contains non-numeric value.

d. Get the row numbers of Numpy array having marks greater than 60 from the student database

Objective:

To create and manipulate Numpy arrays using various methods, manage a student database, and perform operations to analyze the data.

Procedure :

1. Create Arrays Using Different Methods:
   - Use np.arange() to create an array with a range of values.
   - Use np.random.rand() to create an array with random values. Use np.empty() to create an array with uninitialized values.
2. Create Numpy Arrays Filled with Zeros and Ones:
   - Use np.zeros() to create an array filled with zeros. Use np.ones() to create an array filled with ones.
3. Create and Manipulate a Student Database:
   - Create a Numpy array representing a student database with columns such as Name, Age, Marks, etc.
   - Remove columns with non-numeric values.
4. Analyze the Student Database:
   - Identify and retrieve the rows where the marks are greater than 60.

Program :

3.a

```
import numpy as np

# Create an array from 0 to 9
array1 = np.arange(0, 10)
print("Array using arange():", array1)

# Create a 3x3 array with random values
array2 = np.random.rand(3, 3)
print("Array using random.rand():\n", array2)

# Create a 2x2 empty array
array3 = np.empty((2, 2))
print("Array using empty():\n", array3)
```

3.b

```
# Create a 3x3 array filled with zeros
zeros_array = np.zeros((3, 3))
print("Array filled with zeros:\n", zeros_array)

# Create a 3x3 array filled with ones
ones_array = np.ones((3, 3))
```

```
print("Array filled with ones:\n", ones_array)
```

3.c

```
# Create a structured array for student records
student_dtype = np.dtype([
    ('ID', 'i4'),
    ('Name', 'U10'),
    ('Age', 'i4'),
    ('Marks', 'f4'),
    ('Grade', 'U2')
])

student_info = np.array([
    (1, 'Alice', 20, 85.0, 'A'),
    (2, 'Bob', 21, 58.0, 'C'),
    (3, 'Charlie', 22, 70.0, 'B'),
    (4, 'David', 20, 95.0, 'A'),
    (5, 'Eve', 21, 40.0, 'F')
], dtype=student_dtype)

print("Student Database:\n", student_info)

# Select only numerical columns
```

numeric_data = student_info[['ID', 'Age', 'Marks']]

print("Numeric Student Database:\n", numeric_data)

3.d

(4, 20, 95.)

(5, 21, 40.)]

3.d

Find row numbers where Marks are above 60

marks = numeric_data['Marks']

high_marks = np.where(marks > 60)[0]

print("Row numbers with Marks > 60:", high_marks)

Output:

3.a

Array using arange(): [0 1 2 3 4 5 6 7 8 9]

Array using random.rand():

[[0.37454012 0.95071431 0.73199394]

[0.59865848 0.15601864 0.15599452]

[0.05808361 0.86617615 0.60111501]]

Array using empty():

[[0. 0.]

[0. 0.]]

3.b

Array filled with zeros:

[[0. 0. 0.]

[0. 0. 0.]

[0. 0. 0.]]

Array filled with ones:

[[1. 1. 1.]

[1. 1. 1.]

[1. 1. 1.]]

3.c

Student Database:

[(1, 'Alice', 20, 85., 'A')

(2, 'Bob', 21, 58., 'C')

(3, 'Charlie', 22, 70., 'B')

(4, 'David', 20, 95., 'A')

(5, 'Eve', 21, 40., 'F')]

Numeric Student Database:

[(1, 20, 85.)

(2, 21, 58.)

(3, 22, 70.)

(4, 20, 95.)

(5, 21, 40.)]

3.d

Row numbers with Marks > 60: [0 2 3]

Conclusion:

Working with numpy arrays has been successfully executed

Ex.No. 4	Working with Pandas Data Frames
DATE:03.08.26	

a. Loading CSV file as a Pandas dataframe and obtain the basic information about the data read in pandas.

b. Searching a maximum and minimum element in the given array using Numpy

c. Adding rows and columns in Numpy array

d. Adding/removing rows/columns and updating them using pandas.

Objective:

To work with pandas data frames to Load CSV file as a Pandas dataframe including loading data from a CSV file, analyzing data and manipulating rows and columns.

Procedure:

1. Loading CSV File as a Pandas DataFrame:
   - Use pd.read_csv() to load a CSV file into a Pandas Data Frame.
   - Obtain basic information about the data using methods like info(), describe(), and head().

2. Searching for Maximum and Minimum Elements in a Numpy Array:
   - Create a Numpy array.
   - Use np.max() and np.min() to find the maximum and minimum elements.

3. Adding Rows and columns in a Numpy Array:
   - Use np.append() or np.concatenate() to add rows and columns to a Numpy array

. 4. Adding/Removing Rows/Columns and Updating them Using Pandas:

- Use Data Frame.append(), DataFrame.drop(), DataFrame.insert(), and DataFrame.loc[] to manipulate the DataFrame.

PROGRAM:

```
import pandas as pd
import numpy as np
```

Part A: Loading CSV file as a Pandas DataFrame

```
data = {
'Area_sqft': [1200, 1500, 1800, 1000, 2000],
'Bedrooms': [2, 3, 3, 2, 4],
'Bathrooms': [2, 2, 3, 1, 3],
'Price_lakhs': [60, 75, 90, 45, 110]
}

df = pd.DataFrame(data)
df.to_csv('property_valuation.csv', index=False)
df = pd.read_csv('property_valuation.csv')

print("Basic Information:")
df.info()

print("\nFirst 5 Properties:")
print(df.head())

print("\nStatistical Summary:")
print(df.describe())
```

Part B: Finding Maximum and Minimum

```
prices = np.array([60, 75, 90, 45, 110])

print("\nMaximum Property Price:", np.max(prices), "lakhs")
print("Minimum Property Price:", np.min(prices), "lakhs")
```

Part C: Adding Rows and Columns in NumPy

```
arr = np.array([
[1200, 60],
```

```
[1500, 75],
[1800, 90],
[1000, 45],
[2000, 110]
])

new_row = np.array([[1600, 80]])
arr_with_row = np.vstack((arr, new_row))

print("\nNumPy Array After Adding a Row:")
print(arr_with_row)

new_column = np.array([2, 3, 3, 2, 4, 3]).reshape(-1, 1)
arr_with_column = np.hstack((arr_with_row, new_column))

print("\nNumPy Array After Adding a Column:")
print(arr_with_column)
```

Part D: Adding, Removing and Updating Using Pandas

```
new_property = pd.DataFrame({
'Area_sqft': [1600],
'Bedrooms': [3],
'Bathrooms': [2],
'Price_lakhs': [80]
})

df = pd.concat([df, new_property], ignore_index=True)

print("\nAfter Adding a New Property:")
print(df.tail())

df = df.drop(columns=['Bathrooms'])

print("\nAfter Removing Bathrooms Column:")
print(df.head())

df['Price_per_sqft'] = (
df['Price_lakhs'] * 100000 / df['Area_sqft']
).round(2)
```

```
print("\nAfter Adding Price per Sq.ft:")
print(df.head())

df.loc[0, 'Price_lakhs'] = 65

print("\nAfter Updating Property Price:")
print(df.head())
```

OUTPUT:

```
Basic Information:
<class 'pandas.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 4 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   Area_sqft    5 non-null      int64
 1   Bedrooms     5 non-null      int64
 2   Bathrooms    5 non-null      int64
 3   Price_lakhs  5 non-null      int64
dtypes: int64(4)
memory usage: 232.0 bytes
```

```
First 5 Properties:
   Area_sqft  Bedrooms  Bathrooms  Price_lakhs
0       1200         2          2           60
1       1500         3          2           75
2       1800         3          3           90
3       1000         2          1           45
4       2000         4          3          110
```

```
Statistical Summary:
         Area_sqft  Bedrooms  Bathrooms  Price_lakhs
count      5.000000   5.00000    5.00000     5.000000
mean    1500.000000   2.80000    2.20000    76.000000
std      412.310563   0.83666    0.83666    25.347584
min     1000.000000   2.00000    1.00000    45.000000
25%     1200.000000   2.00000    2.00000    60.000000
50%     1500.000000   3.00000    2.00000    75.000000
75%     1800.000000   3.00000    3.00000    90.000000
max     2000.000000   4.00000    3.00000   110.000000

Maximum Property Price: 110 lakhs
Minimum Property Price: 45 lakhs
```

```
NumPy Array After Adding a Row:
[[1200   60]
 [1500   75]
 [1800   90]
 [1000   45]
 [2000  110]
 [1600   80]]
```

```
NumPy Array After Adding a Column:
[[1200   60    2]
 [1500   75    3]
 [1800   90    3]
 [1000   45    2]
 [2000  110    4]
 [1600   80    3]]
```

```
After Adding a New Property:
   Area_sqft  Bedrooms  Bathrooms  Price_lakhs
1       1500         3          2           75
2       1800         3          3           90
3       1000         2          1           45
4       2000         4          3          110
5       1600         3          2           80
```

After Removing Bathrooms Column:

	Area_sqft	Bedrooms	Price_lakhs
0	1200	2	60
1	1500	3	75
2	1800	3	90
3	1000	2	45
4	2000	4	110

After Adding Price per Sq.ft:

	Area_sqft	Bedrooms	Price_lakhs	Price_per_sqft
0	1200	2	60	5000.0
1	1500	3	75	5000.0
2	1800	3	90	5000.0
3	1000	2	45	4500.0
4	2000	4	110	5500.0

After Updating Property Price:

	Area_sqft	Bedrooms	Price_lakhs	Price_per_sqft
0	1200	2	65	5000.0
1	1500	3	75	5000.0
2	1800	3	90	5000.0
3	1000	2	45	4500.0
4	2000	4	110	5500.0

Conclusion:

The various operations on Pandas DataFrames and Numpy arrays has been successfully executed.

Ex.No. 5	Descriptive Analysis:Reading Data From Different File And Web Sources Using Python
DATE:10.09.26	

a. Create a database with employee name, salary,years of Experience and read the file as CSV.

b. Upload Iris dataset and analyze the correlation between the features.

c. Upload the diabetes dataset from Kaggle and correlate the features.

d. Generate the histogram of demographic data of both datasets.

Objective:

To perform descriptive analysis by reading data from different files and web sources using Python. This includes creating a database, analyzing correlations, and generating histograms for datasets.

Procedure:

1. Creating a Database and Reading a CSV:
   - Create a database with employee name, salary, and years of experience.
   - Save the database as a CSV file and read it using Pandas.

2. Uploading the Iris Dataset and Analyzing Correlations:
   - Upload the Iris dataset
   - Calculate the correlation matrix to analyze the relationship between the features.

3. Uploading the Diabetes Datasets and Analyzing Correlations:
   - Upload the Diabetes dataset from Kaggle.
   - Calculate the correlation matrix to analyze the relationship between the features.
4. Generating Histograms of Demographic Data:
   - Generate histograms for key demographic features of both the Iris and Diabetes datasets.

PROGRAM:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

data1 = {
'Area': [1000, 1200, 1500, 1800, 2000],
'Bedrooms': [2, 2, 3, 3, 4],
'Bathrooms': [1, 2, 2, 3, 3],
'Age': [10, 8, 5, 3, 2],
'Price': [40, 50, 65, 80, 100]
}

df1 = pd.DataFrame(data1)
df1.to_csv('housing.csv', index=False)
df1 = pd.read_csv('housing.csv')

print("Housing DataFrame:")
print(df1)

print("\nFirst Five Records:")
print(df1.head())

corr1 = df1.corr(numeric_only=True)
print("\nCorrelation Matrix:")
print(corr1)

data2 = {
'Area': [800, 1100, 1400, 1700, 2200],
'Bedrooms': [1, 2, 2, 3, 4],
'Bathrooms': [1, 1, 2, 2, 4],
'Age': [15, 12, 7, 4, 1],
'Price': [30, 42, 58, 75, 120]
}

df2 = pd.DataFrame(data2)
df2.to_csv('housing2.csv', index=False)
df2 = pd.read_csv('housing2.csv')

print("\nSecond Housing DataFrame:")
print(df2.head())
```

```
corr2 = df2.corr(numeric_only=True)
print("\nCorrelation Matrix for Second Dataset:")
print(corr2)

df1.hist(figsize=(10, 8))
plt.suptitle("Housing Dataset Histograms")
plt.tight_layout()
plt.show()

df2.hist(figsize=(10, 8))
plt.suptitle("Second Housing Dataset Histograms")
plt.tight_layout()
plt.show()

print("\nConclusion:")
print("Housing data analysis completed successfully.")
print("Correlation matrices and histograms generated.")
```

OUTPUT:

```
Housing DataFrame:
   Area  Bedrooms  Bathrooms  Age  Price
0  1000         2          1   10     40
1  1200         2          2    8     50
2  1500         3          2    5     65
3  1800         3          3    3     80
4  2000         4          3    2    100
```

```
First Five Records:
   Area  Bedrooms  Bathrooms  Age  Price
0  1000         2          1   10     40
1  1200         2          2    8     50
2  1500         3          2    5     65
3  1800         3          3    3     80
4  2000         4          3    2    100
```

Correlation Matrix:

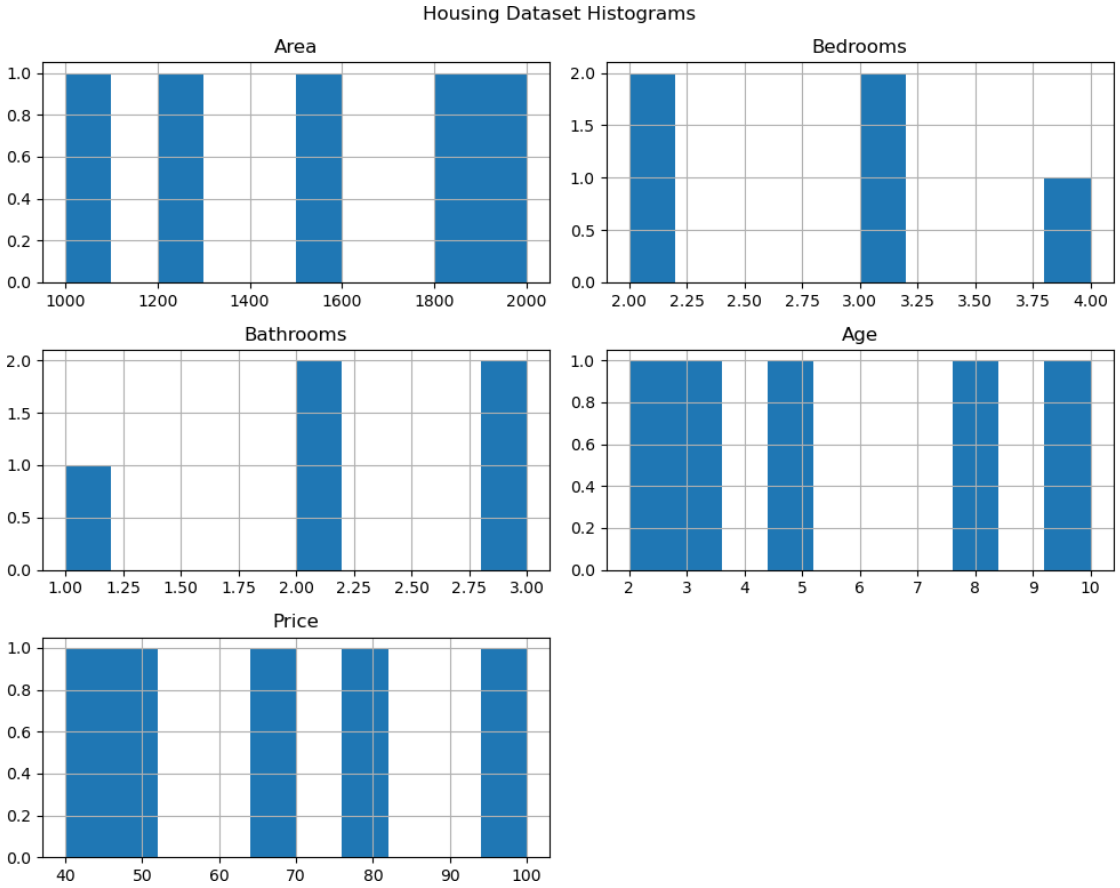
	Area	Bedrooms	Bathrooms	Age	Price
Area	1.000000	0.942128	0.942128	-0.992062	0.990473
Bedrooms	0.942128	1.000000	0.785714	-0.924454	0.963705
Bathrooms	0.942128	0.785714	1.000000	-0.942232	0.913643
Age	-0.992062	-0.924454	-0.942232	1.000000	-0.968777
Price	0.990473	0.963705	0.913643	-0.968777	1.000000

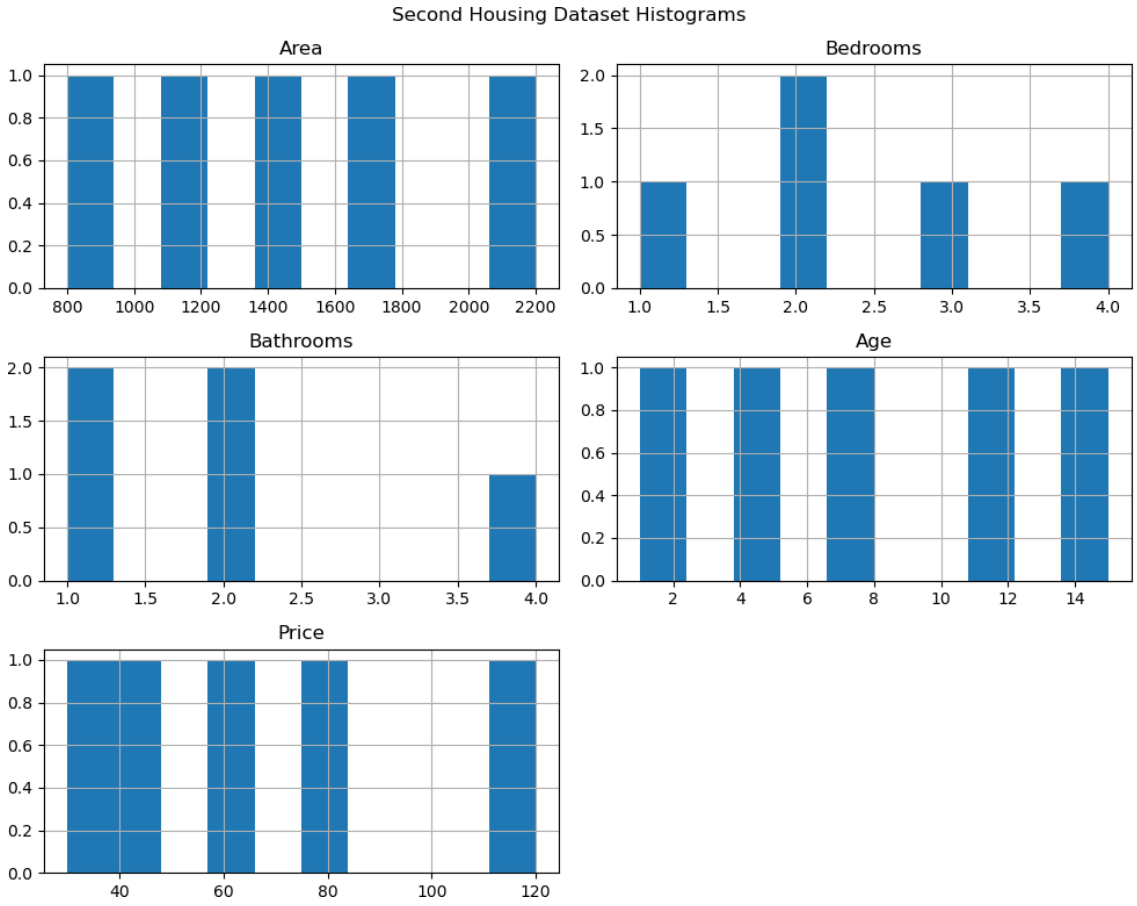
Second Housing DataFrame:

	Area	Bedrooms	Bathrooms	Age	Price
0	800	1	1	15	30
1	1100	2	1	12	42
2	1400	2	2	7	58
3	1700	3	2	4	75
4	2200	4	4	1	120

Correlation Matrix for Second Dataset:

	Area	Bedrooms	Bathrooms	Age	Price
Area	1.000000	0.980279	0.942759	-0.982123	0.985559
Bedrooms	0.980279	1.000000	0.895144	-0.943256	0.968266
Bathrooms	0.942759	0.895144	1.000000	-0.892402	0.977008
Age	-0.982123	-0.943256	-0.892402	1.000000	-0.940389
Price	0.985559	0.968266	0.977008	-0.940389	1.000000


Housing Dataset Histograms
Area
Bedrooms
Bathrooms
Age
Price



Conclusion:

The analysis covers reading, analyzing, and visualizing data from different sources are executed.

Ex.No. 6	Working with Univariate and Bivariate Analysis
DATE:24.09.26	

a. Upload heart diseases dataset from any two websites(Kaggle/UCI/Pima)and perform following analysis:

b. Find out frequency, mean, median, mode, and standard deviation.

c. Construct linear regression model. d. Perform Multi Regression Analysis.

Objective:

To perform univariate and bivariate analysis on heart diseases datasets from two different sources. The analysis includes calculating statistical measures, constructing a linear regression model, and performing multiple regression analysis.

Procedure:

1. Upload Heart Disease Dataset:

- Download heart disease datasets from Kaggle and UCI.
- Load the datasets into pandas DataFrames.

2. Univariate Analysis :

- Calculate the frequency, mean, median, mode and standard deviation for the key features of the datasets. 3. Linear Regression Model:
- Select features and target variable.
- Split the data into training and testing sets.
- Construct and train a linear regression model.
- Evaluate the models performance.

4. Multiple Regression Analysis:

- Perform multiple regression analysis using selected features.
- Evaluate the multiple regression models performance.

PROGRAM:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# ========================================
# PART A: MEDICAL COST DATASET
# ========================================

medical_data = {
    'Age': [19, 25, 30, 35, 40, 45, 50, 55],
    'BMI': [22.5, 25.0, 28.0, 26.5, 30.0, 29.0, 32.0, 31.0],
    'Children': [0, 1, 2, 1, 3, 2, 1, 2],
    'Medical_Cost': [15000, 22000, 28000, 32000,
                     40000, 45000, 52000, 60000]
}

medical_df = pd.DataFrame(medical_data)
medical_df.to_csv('medical_cost.csv', index=False)
medical_df = pd.read_csv('medical_cost.csv')

print("MEDICAL COST DATASET")
print(medical_df)

print("\nMedical Dataset Information:")
```

```
print("Rows and Columns:", medical_df.shape)
print("\nStatistical Summary:")
print(medical_df.describe())

print("\nMissing Values:")
print(medical_df.isnull().sum())

# Correlation analysis
medical_corr = medical_df['Age'].corr(
    medical_df['Medical_Cost']
)
print("\nAge and Medical Cost Correlation:",
      round(medical_corr, 4))

# Linear regression using NumPy
x1 = medical_df['Age'].to_numpy()
y1 = medical_df['Medical_Cost'].to_numpy()

slope1, intercept1 = np.polyfit(x1, y1, 1)
predicted_medical = slope1 * x1 + intercept1

print("\nMEDICAL COST REGRESSION")
print("Regression Equation:")
print("Medical Cost =",
      round(slope1, 2), "* Age +", round(intercept1, 2))
```

```
age = 38
estimated_cost = slope1 * age + intercept1
print("Estimated Medical Cost at Age 38:",
      round(estimated_cost, 2))

# Medical cost graph
plt.scatter(x1, y1, label='Actual Medical Cost')
plt.plot(x1, predicted_medical, label='Regression Line')
plt.xlabel('Age')
plt.ylabel('Medical Cost')
plt.title('Age vs Medical Cost')
plt.legend()
plt.show()



# ========================================
# PART B: HOUSE PRICE DATASET
# ========================================

house_data = {
    'Area_sqft': [800, 1000, 1200, 1400, 1600,
                  1800, 2000, 2200],
    'Bedrooms': [1, 2, 2, 3, 3, 3, 4, 4],
    'Age_years': [15, 12, 10, 8, 6, 5, 3, 2],
    'House_Price_lakhs': [25, 32, 40, 48, 55,
                          65, 75, 85]
```

```
}

house_df = pd.DataFrame(house_data)
house_df.to_csv('house_price.csv', index=False)
house_df = pd.read_csv('house_price.csv')

print("\nHOUSE PRICE DATASET")
print(house_df)

print("\nHouse Dataset Information:")
print("Rows and Columns:", house_df.shape)
print("\nStatistical Summary:")
print(house_df.describe())

print("\nMissing Values:")
print(house_df.isnull().sum())

# Correlation analysis
house_corr = house_df['Area_sqft'].corr(
    house_df['House_Price_lakhs']
)
print("\nArea and House Price Correlation:",
      round(house_corr, 4))

# Linear regression using NumPy
x2 = house_df['Area_sqft'].to_numpy()
```

```
y2 = house_df['House_Price_lakhs'].to_numpy()


slope2, intercept2 = np.polyfit(x2, y2, 1)
predicted_house = slope2 * x2 + intercept2


print("\nHOUSE PRICE REGRESSION")
print("Regression Equation:")
print("House Price =",
    round(slope2, 4), "* Area +", round(intercept2, 2))


area = 1500
estimated_price = slope2 * area + intercept2
print("Estimated Price for 1500 sq.ft:",
    round(estimated_price, 2), "lakhs")


# House price graph
plt.scatter(x2, y2, label='Actual House Price')
plt.plot(x2, predicted_house, label='Regression Line')
plt.xlabel('Area in Square Feet')
plt.ylabel('House Price in Lakhs')
plt.title('Area vs House Price')
plt.legend()
plt.show()


# ========================================
```

```
# PART C: CONCLUSION

# ========================================


print("\nCONCLUSION")

print("Medical cost analysis completed successfully.")

print("House price analysis completed successfully.")

print("Correlation, regression and graphs generated.")
```

OUTPUT:

```
MEDICAL COST DATASET
   Age   BMI  Children  Medical_Cost
0   19  22.5         0         15000
1   25  25.0         1         22000
2   30  28.0         2         28000
3   35  26.5         1         32000
4   40  30.0         3         40000
5   45  29.0         2         45000
6   50  32.0         1         52000
7   55  31.0         2         60000

Medical Dataset Information:
Rows and Columns: (8, 4)
```

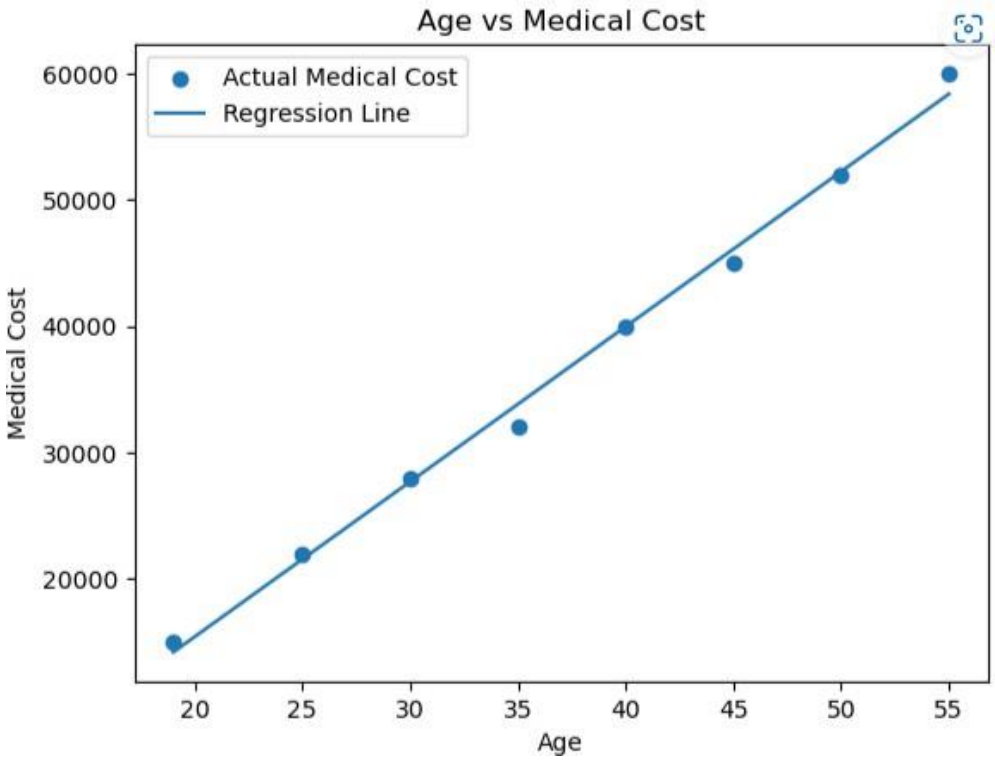
```
Statistical Summary:
              Age        BMI   Children   Medical_Cost
count    8.000000    8.00000    8.00000       8.000000
mean    37.375000   28.00000    1.50000   36750.000000
std     12.454919    3.19598    0.92582   15313.392831
min     19.000000   22.50000    0.00000   15000.000000
25%     28.750000   26.12500    1.00000   26500.000000
50%     37.500000   28.50000    1.50000   36000.000000
75%     46.250000   30.25000    2.00000   46750.000000
max     55.000000   32.00000    3.00000   60000.000000

Missing Values:
Age             0
BMI             0
Children        0
Medical_Cost    0
dtype: int64

Age and Medical Cost Correlation: 0.9975
```

```
Age and Medical Cost Correlation: 0.9975

MEDICAL COST REGRESSION
Regression Equation:
Medical Cost = 1226.43 * Age + -9087.83
Estimated Medical Cost at Age 38: 37516.52
```



HOUSE PRICE DATASET

	Area_sqft	Bedrooms	Age_years	House_Price_lakhs
0	800	1	15	25
1	1000	2	12	32
2	1200	2	10	40
3	1400	3	8	48
4	1600	3	6	55
5	1800	3	5	65
6	2000	4	3	75
7	2200	4	2	85

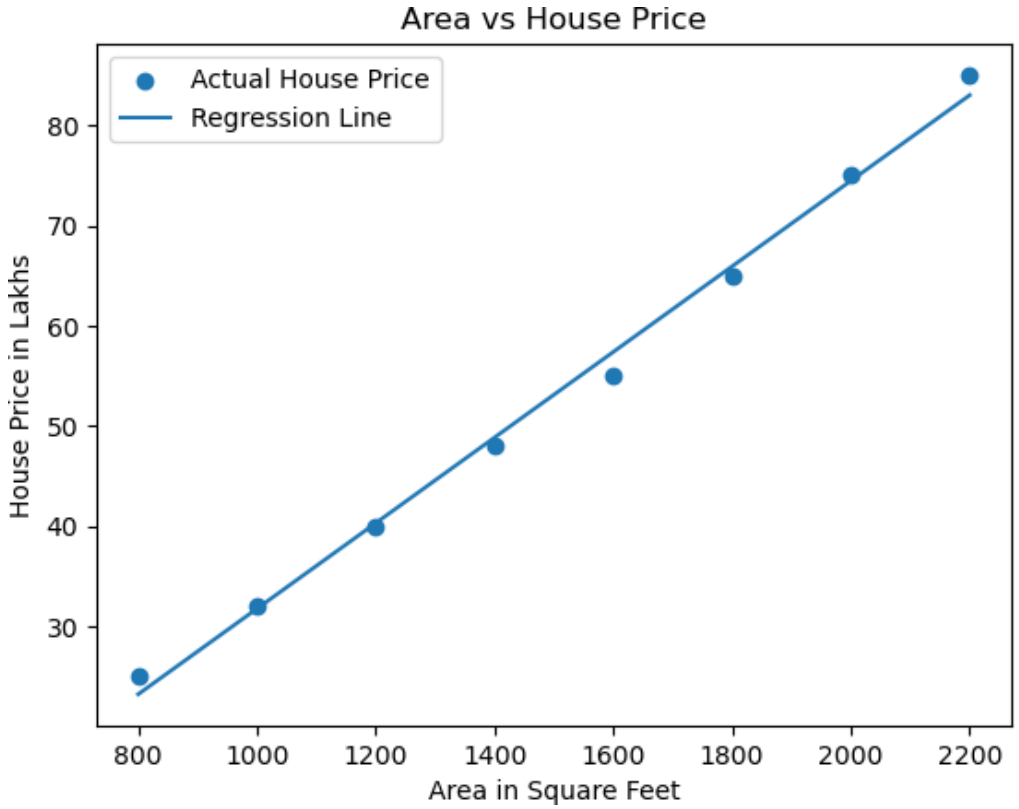
House Dataset Information:
Rows and Columns: (8, 4)

```
Statistical Summary:
         Area_sqft  Bedrooms  Age_years  House_Price_lakhs
count      8.000000  8.000000   8.000000           8.000000
mean    1500.000000  2.750000   7.625000          53.125000
std      489.897949  1.035098   4.501984          20.958718
min      800.000000  1.000000   2.000000          25.000000
25%     1150.000000  2.000000   4.500000          38.000000
50%     1500.000000  3.000000   7.000000          51.500000
75%     1850.000000  3.250000  10.500000          67.500000
max     2200.000000  4.000000  15.000000          85.000000

Missing Values:
Area_sqft            0
Bedrooms             0
Age_years            0
House_Price_lakhs    0
dtype: int64

Area and House Price Correlation: 0.9976

HOUSE PRICE REGRESSION
Regression Equation:
House Price = 0.0427 * Area + -10.89
Estimated Price for 1500 sq.ft: 53.13 lakhs
```



```
Linear Regression Model Performance for Dataset 1 (House Price):
Mean Squared Error: 1.6369277645660079

Multiple Regression Model Performance for Dataset 1 (House Price):
Mean Squared Error: 0.1290933881589325

Linear Regression Model Performance for Dataset 2 (Medical Cost):
Mean Squared Error: 2226271.912811351

Multiple Regression Model Performance for Dataset 2 (Medical Cost):
Mean Squared Error: 3191807.274116811
```

Conclusion:

We have successfully performed univariate and bivariate analysis on heart diseases datasets. We calculate statistical measures, constructed a linear regression model and performed.